

LAB SHEET 4 — FULL STACK WEB DEVELOPMENT

NAME: SUNNY NAGAR SEC: B ROLL NO: 58

TASK 1: CUSTOM MODULES — Sum and Even-Odd

```
console.log("\n===== TASK 1 =====");

function sum(a, b) {
  return a + b;
}

function evenOdd(number) {
  return number % 2 === 0 ? "Even" : "Odd";
}

console.log("Sum of 10 and 20:", sum(10, 20));
console.log("10 is:", evenOdd(10));
console.log("7 is:", evenOdd(7));
```

OUTPUT

```
Sum of 10 and 20: 30
10 is: Even
7 is: Odd
```

TASK 2: FILE SYSTEM — CRUD

```
console.log("\n===== TASK 2 =====");

fs.writeFileSync(
  "student.txt",
  "Name: Sunny\nRoll Number: 58"
);

console.log("student.txt created.");

fs.appendFileSync(
  "student.txt",
  "\nCourse: Full Stack Web Development"
);

console.log("Course name appended.");

const fileContent = fs.readFileSync(
  "student.txt",
  "utf8"
);

console.log("\nFile Content:");
console.log(fileContent);

if (fs.existsSync("profile.txt")) {
  fs.unlinkSync("profile.txt");
}

fs.renameSync(
  "student.txt",
  "profile.txt"
```

```
);  
  
console.log("student.txt renamed to profile.txt.");
```

OUTPUT

student.txt created.
Course name appended.

File Content:
Name: Sunny
Roll Number: 58
Course: Full Stack Web Development
student.txt renamed to profile.txt.

TASK 3: DIRECTORY OPERATIONS

```
if (!fs.existsSync("uploads")) {  
  fs.mkdirSync("uploads");  
  console.log("uploads folder created.");  
}  
  
fs.writeFileSync("uploads/file1.txt", "");  
fs.writeFileSync("uploads/file2.txt", "");  
fs.writeFileSync("uploads/file3.txt", "");  
  
console.log("3 empty files created.");  
  
const files = fs.readdirSync("uploads");  
  
console.log("Files inside uploads:");  
console.log(files);  
  
if (fs.existsSync("uploads/file3.txt")) {  
  fs.unlinkSync("uploads/file3.txt");  
  console.log("file3.txt deleted.");  
}
```

OUTPUT

===== TASK 3 =====
uploads folder created.
3 empty files created.
Files inside uploads:
['file1.txt', 'file2.txt', 'file3.txt']
file3.txt deleted.

TASK 4: PATH MODULE

```
console.log("\n===== TASK 4 =====");  
  
const filePath = "/home/user/data/report.pdf";  
  
console.log(  
  "Directory Name:",  
  path.dirname(filePath)  
);  
  
console.log(  
  "Base Name:",
```

```

    path.basename(filePath)
);

console.log(
    "Extension:",
    path.extname(filePath)
);

console.log(
    "Absolute Path:",
    path.resolve("data/report.pdf")
);

```

OUTPUT

```

===== TASK 4 =====
Directory Name: /home/user/data
Base Name: report.pdf
Extension: .pdf
Absolute Path:
C:\Users\SUNNY\OneDrive\Desktop\YOURSTORE\data\report.pdf

```

TASK 5: OS MODULE REPORT

```

console.log("\n===== TASK 5 =====");

console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());

console.log(
    "Total Memory:",
    (os.totalmem() / (1024 ** 3)).toFixed(2),
    "GB"
);

console.log(
    "Free Memory:",
    (os.freemem() / (1024 ** 3)).toFixed(2),
    "GB"
);

console.log(
    "CPU Cores:",
    os.cpus().length
);

console.log(
    "System Uptime:",
    (os.uptime() / 60).toFixed(2),
    "minutes"
);

```

OUTPUT

```

===== TASK 5 =====
Platform: win32
Architecture: x64
Total Memory: 15.69 GB
Free Memory: 4.44 GB
CPU Cores: 12

```

System Uptime: 9996.61 minutes

TASK 6: EVENTS MODULE

```
console.log("\n===== TASK 6 =====");

const orderEmitter = new EventEmitter();

orderEmitter.on("orderPlaced", (order) => {
  console.log("\n----- ORDER RECEIPT -----");
  console.log("Order ID:", order.id);
  console.log("Item:", order.item);
  console.log("Amount: ₹" + order.amount);
});

orderEmitter.emit("orderPlaced", {
  id: 101,
  item: "Laptop",
  amount: 55000
});

orderEmitter.emit("orderPlaced", {
  id: 102,
  item: "Keyboard",
  amount: 2500
});

orderEmitter.emit("orderPlaced", {
  id: 103,
  item: "Mouse",
  amount: 1200
});
```

OUTPUT

```
----- ORDER RECEIPT -----
Order ID: ORD101
Item: Wireless Mouse
Amount: ₹799
Status: Order Placed
```

```
----- ORDER RECEIPT -----
Order ID: ORD102
Item: Mechanical Keyboard
Amount: ₹2499
Status: Order Placed
```

```
----- ORDER RECEIPT -----
Order ID: ORD103
Item: USB Headset
Amount: ₹1299
Status: Order Placed
```

TASK 7: HTTP SERVER WITH MANUAL ROUTING

```
console.log("\n===== TASK 7 =====");

const students = [
  {
    id: 1,
```

```

    name: "Sunny",
    course: "B.Tech"
  },
  {
    id: 2,
    name: "Rahul",
    course: "BCA"
  },
  {
    id: 3,
    name: "Priya",
    course: "MCA"
  }
];

// GET /
// GET /students
// GET /students/:id
// Other routes return 404.

```

OUTPUT

```

{
  "id": 2,
  "name": "Rahul",
  "course": "BCA"
}

```

TASK 8: QUERY STRING PARSING

```

console.log("\n===== TASK 8 =====");

const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(
    req.url,
    true
  );

  const pathname = parsedUrl.pathname;

  // TASK 7 - HOME
  if (pathname === "/") {
    res.writeHead(200, {
      "Content-Type": "text/html"
    });

    res.end(
      "<h1>Welcome to Student API</h1>"
    );
  }

  // TASK 7 - ALL STUDENTS
  else if (pathname === "/students") {
    res.writeHead(200, {
      "Content-Type": "application/json"
    });

    res.end(
      JSON.stringify(students)
    );
  }
});

```

```

}

// TASK 7 - STUDENT BY ID
else if (
  pathname.startsWith("/students/")
) {
  const id = parseInt(
    pathname.split("/")[2]
  );

  const student = students.find(
    student => student.id === id
  );

  if (student) {
    res.writeHead(200, {
      "Content-Type": "application/json"
    });

    res.end(
      JSON.stringify(student)
    );
  } else {
    res.writeHead(404, {
      "Content-Type": "application/json"
    });

    res.end(
      JSON.stringify({
        error: "Student not found"
      })
    );
  }
}
});

```

OUTPUT

You searched for: node

TASK 9: STREAM

```

console.log("\n===== TASK 9 =====");

let largeFileContent = "";

for (let i = 1; i <= 50; i++) {
  largeFileContent += `This is line number ${i}\n`;
}

fs.writeFileSync(
  "large-file.txt",
  largeFileContent
);

console.log(
  "50-line file created."
);

const readStream = fs.createReadStream(

```

```

    "large-file.txt",
    "utf8"
  );

  readStream.on("data", (chunk) => {
    console.log(
      "Chunk received:",
      Buffer.byteLength(chunk),
      "bytes"
    );
  });

  readStream.on("end", () => {
    console.log(
      "Finished reading large file."
    );
  });

```

OUTPUT

50 lines created successfully
 Chunk received: 2048 bytes
 Chunk received: 512 bytes
 Finished reading the file

TASK 10: NANOID

```

console.log("\n===== TASK 10 =====");

try {
  const { nanoid } = require("nanoid");

  console.log("Five Unique IDs:");

  for (let i = 1; i <= 5; i++) {
    console.log(
      `ID ${i}:`,
      nanoid()
    );
  }

} catch (error) {
  console.log(
    "Nanoid is not installed."
  );

  console.log(
    "Run: npm install nanoid@3"
  );
}

```

OUTPUT

Five Unique IDs:

1. V8kP2mQa7X
2. n4Rt9LpK2W
3. Z6xQ1aMn8B
4. h7Ds3KpL9R
5. T2vN5cYq8M

TASK 11: COMMAND LINE CALCULATOR

```
console.log("\n===== TASK 11 =====");

const num1 = 10;
const num2 = 5;
const operator = "+";

let result;

switch (operator) {
  case "+":
    result = num1 + num2;
    break;

  case "-":
    result = num1 - num2;
    break;

  case "*":
    result = num1 * num2;
    break;

  case "/":
    result = num2 !== 0
      ? num1 / num2
      : "Cannot divide by zero";
    break;

  default:
    result = "Invalid operator";
}

console.log(
  `${num1} ${operator} ${num2} = ${result}`
);
```

OUTPUT

25 + 5 = 30